

Theoretical Definition:

A **Singleton** is a **design pattern** that ensures a class has **only one instance** throughout the lifetime of a program and provides a **global point of access** to that instance.

Why Use a Singleton?

- To avoid duplication of **heavy resources** (like a database connection).
- To **share global state or configuration**.
- To maintain **consistent behavior** across the application.

Classic Singleton Example in Python:

python

 Copiar

 Editar

```
class Singleton:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            print("Creating new instance")
            cls._instance = super(Singleton, cls).__new__(cls)
        return cls._instance

# Test the class
s1 = Singleton()
s2 = Singleton()

print(s1 is s2) # True - both variables point to the same object
```



Explanation:

- `__new__` is the method that creates an instance of the class.
- It checks if an instance already exists (`_instance`).
- If not, it creates and stores one.
- If it exists, it returns the same instance.



Alternative Using a Decorator (More Advanced Version):

python

📄 Copiar

✎ Editar

```
def singleton(cls):
    instances = {}
    def wrapper(*args, **kwargs):
        if cls not in instances:
            instances[cls] = cls(*args, **kwargs)
        return instances[cls]
    return wrapper

@singleton
class Config:
    def __init__(self):
        self.value = 42

a = Config()
b = Config()

print(a is b)  # True
```